From netlist to verified model

Automatic real-number-model and testbench generation

for mixed-signal verification

Veylon Systems

Abstract

An analog block is slow to simulate. SPICE has to solve the physics of every transistor in it. A real-number model is the shortcut: it takes the input number, does a little ordinary arithmetic, and hands back the output number. No physics, no transistors. That lets it run in a normal digital simulator, right beside the chip's logic, fast enough to sit in a full regression suite.

The catch is the writing. Someone has to build the model, keep it in step with the circuit as the circuit changes, and show the team that it still matches. The first two jobs are slow. The third one usually never happens at all.

spice2rnm does all three from one command. You hand it a SPICE netlist. You get back a SystemVerilog real-number model, a UVM-MS verification environment that tests the model, and evidence that both agree with the real circuit.

Why believe the evidence? Three rules. First, every expected value in every generated testbench is a *SPICE measurement of your circuit*. Not one of them is computed from the model's own equations, so a test cannot pass just by agreeing with itself. Second, every generated environment is shown to be able to fail: the tool breaks the model on purpose and confirms that the test then fails. Third, each block is checked on what it is for. A filter on its frequency response. A duty-cycle corrector on where its clock edges land. A phase interpolator on how far it shifts the clock at each setting.

The paper works through four circuits, easiest first. The last one, a PLL, gets run twice. Run one keeps the transistors: the digital RTL runs in the SystemVerilog simulator while the analog blocks run as the transistor-level circuit in SPICE, with a bridge joining the two. That run checks the generated models against the real circuit. Run two drops SPICE entirely: the RTL and the generated models, nothing else. Every number in this paper came out of the shipped tool, in about an hour on a laptop.

On one page

What you give it. A SPICE netlist, flat or with subcircuits. Optionally, a plain-language specification as well, and an existing model library whose conventions the tool should follow. You do not have to label ports, supply a model template or write a testbench.

What you get back, from one command. Three things:

- a SystemVerilog real-number model, whose structure the tool picks from measurements of your circuit;
- a UVM-MS environment that tests the model, in which every expected value is a SPICE measurement of your circuit;
- evidence that the model matches the circuit. Circuit and model get identical inputs, their outputs are compared in the way that matters for that block, and the waveforms are saved.

Results on five circuits. One command each. All five together took about an hour on a laptop, on open-source simulators.

Circuit	What is compared	Worst vs pass/fail limit	Environment
two-pole filter	waveform, unseen input	0.0097 vs 0.15	64 checks, 0 failed
duty-cycle corrector	where its edges land	0.428 vs 1.0 pp	60 checks, 0 failed
phase interpolator	edge position per code	1.83 vs 6.25 ps	9 checks, 0 failed
PLL, SV–SPICE co-sim	system vs SPICE circuit	4.158 mV, 47% of limit	5 blocks, 0 failed
PLL, SV only	loop closed in SV alone	locked, −0.0005%	12 checks, 0 failed

Why a passing result here is worth believing. Three rules hold throughout.

- The expected answers come from SPICE, never from the model. A test cannot pass by agreeing with itself.
- Every environment is shown to be able to fail. The tool breaks the model on purpose, confirms the test fails, and measures the smallest break the test still catches.
- Each block is checked on its own job: a filter on its frequency response, a clock buffer on its edges, an oscillator on its frequency. A check that does not apply to a block is reported as "not applicable", never counted as a pass.

Where the evidence is. Every model, environment and verdict in this paper is published exactly as the tool wrote it, at `https://github.com/vrajeshprakhya/spice2rnm-demos`. One command regenerates all of it. The outputs are plain IEEE 1800 SystemVerilog with no vendor dependence. Here they ran on ngspice and on the open-source simulator xezim, and they will run on the simulators your flow already uses.

Contents

Terms used in this paper 3

1 The problem 4

2 What you get from one command 4

3 Case study 1: two-pole filter 5

4 Case study 2: duty-cycle corrector 7
4.1 A model that follows how fast the circuit really is . . . 7
4.2 Verified by function, not by formula . . . 7

5 Case study 3: Phase interpolator 8

6 Case study 4: Phase-locked loop 9
6.1 Checked as a whole system, against the transistors . . . 10
6.2 The noise the netlist declares is the noise the model gets . . . 12
6.3 Where the block works is measured, not assumed . . . 13
6.4 The same loop, in SystemVerilog alone . . . 14

7 Results at a glance 20
7.1 What the equivalence folders hold . . . 22

8 What the equivalence check compares 23
8.1 What keeps a figure from flattering itself . . . 24
8.2 Blocks, and then the system . . . 24
8.3 One thing a score at an operating point cannot see . . . 25

9 Why a green testbench is worth believing 26

10 Reading the written specification 27

11 Joining an existing model library 28

12 AI where it helps, physics where it decides 29

13 Scope and requirements 29

A What the routes measure 32
A.1 Why an oscillator is compared as a rate . . . 32
A.2 The tuning curve . . . 32
A.3 Control states . . . 33
A.4 The state space . . . 34

Terms used in this paper

Real-number model (RNM) A SystemVerilog model of an analog block. It works out output values from input values with ordinary arithmetic, so it runs in a digital simulator.

Netlist The SPICE description of a circuit: what parts it has and how they connect. (Older SPICE usage calls it a *deck*, and the tool's own generated files still use that word.)

The transistors Short for the transistor-level circuit in SPICE (here, ngspice): the real circuit, simulated in full. Every model is compared against it. This paper says "the transistors" throughout.

SV–SPICE co-simulation One simulation with a foot in each camp. The design's digital RTL runs in the SystemVerilog simulator while its analog part runs as the transistors in SPICE, and a bridge passes signals between the two. In this paper, "co-simulation" always means that.

SV-only system simulation Also "SystemVerilog only". The design's RTL and the generated models of its analog blocks all run in the SystemVerilog simulator. No SPICE anywhere.

Generated Written as output by spice2rnm. A file "not generated" is one the tool deliberately does not write.

Equivalence check Giving the circuit (in SPICE) and the model the same input, and comparing what comes out.

Golden value An expected value in a testbench. It is measured from the circuit in SPICE, never taken from the model.

Route The measuring method the tool picks for a block: say, measuring a filter's frequency response, or an oscillator's frequency against its control voltage (its tuning curve).

Bar Short for the pass/fail limit a result is compared against.

Verdict The outcome of a check: pass or fail.

Reported for information A number shown for information, with no pass/fail limit applied to it.

rms Root mean square: the standard way of measuring the size of a noise signal or an error.

pp Percentage points (of duty cycle, for example).

Random sample, fixed random seed A simulation with noise in it gives a different answer each run, so one run's answer is one random sample of the possible answers. Fixing the random seed makes the random number generator repeat itself, so a rerun gives the same answer.

White and flicker noise White noise has equal power at all frequencies. Flicker (or 1/f) noise gets stronger towards low frequencies.

Jitter Variation in when clock edges arrive. *Period jitter* is the variation in the time from one edge to the next.

Composed model The individual block models wired together into a model of the whole circuit.

UVM-MS Accellera's Universal Verification Methodology for mixed-signal designs, built on IEEE 1800 SystemVerilog.

`wreal`, nettype Two ways of declaring nets that carry analog (real) values in SystemVerilog. `wreal` comes from Verilog-AMS; a user-defined nettype is the IEEE 1800 equivalent.

`MANIFEST.txt` The list of large output files that are not stored in the repository. For each one it gives the size and the command that recreates it. Such a file is "listed in `MANIFEST.txt`".

1 The problem

A model of an analog block has to be both fast and accurate, and the two pull against each other. Fast, or it cannot run in a digital regression suite at all. Accurate, or passing its tests

means nothing.

Hand-written models drift away from the circuits they stand for, and nothing announces the drift. The model still compiles. It still simulates. It still passes its tests. It has simply stopped describing the circuit.

The usual safeguard is a hand-written testbench, and it has the same problem one level up. A person decides what to check and how close counts as close enough, and those decisions are rarely written down next to the numbers they produced. When the circuit changes, the model, the testbench and the tolerances all need revisiting by hand. In practice that does not happen.

spice2rnm regenerates all three from the netlist whenever you ask. The model, the environment that checks it and the evidence all come out of the same command. So they cannot quietly fall out of step with the circuit, or with each other.

2 What you get from one command

```
spice2rnm dcc.cir --emit-uvm-ms --spec dcc_spec.md
```

You do not name the ports. The tool finds them in the netlist. The supplies carry only a DC level. The input is the node driven by the one source that carries a transient waveform or an AC magnitude. On this netlist that node is `clk`.

Finding the output is a harder problem, because the wiring alone does not say which node you care about. The netlist's own `.save`, `.print` or `.probe` statement does say it: that is the author naming the signal the netlist exists to show. When a netlist has one, the tool takes the output from it. When it has none, as here, the tool guesses the output from the node names. Either way the run reports which of the two it did, so you can tell a declared output from a guessed one.

Four routes never guess: the ones that measure the edge, frequency or current of one particular node. For those, you must name the output.

The command produces three things.

- **A SystemVerilog real-number model.** The tool picks the model's structure to match the block's measured behaviour. That includes blocks whose speed changes with signal level, where any fixed-response model is wrong over most of its range.

  The model has `wreal` ports, so it connects straight into an RNM library alongside the models already there. The model itself connects. There is no wrapper around it.

  That choice affects function, not just style. A `real` port is a *variable*, and a variable takes exactly one driver. Nothing else can share the node, and a bridge cannot attach without a converter module in between. A `wreal` port is a *net*, so several drivers can combine on it: an offset source, a noise injector, a second bridge. The model then drops into a library with nothing standing between it and the nets it is wired to.

  There is one real cost. `wreal` comes from Verilog-AMS, not IEEE 1800, so the model compiles only on simulators that support it, and everything that instantiates the model inherits that requirement. For a flow without `wreal`, `--model-ports real` generates the same model with variable ports, and it depends on no package, no nettype and no language extension. `--model-ports house` instead gives the model your own library's nettype (Section 11).

  The arithmetic is identical in every case. Only the port declarations differ. One detail: a net cannot be assigned inside procedural code, so a model with net ports drives its output through a single continuous assignment. We measured whether that matters. It does not. The filter scores `rms_error_norm` 0.029008790476737805 against SPICE with either port type, identical to every digit.

- **A UVM-MS verification environment** that follows the Accellera UVM-MS specification. It is built on IEEE 1800, not on Verilog-AMS.

  Analog values travel on *user-defined nettypes* (IEEE 1800-2017 §6.6.7). The generated package declares the nettype together with a generated resolution function, `sum_res`, which adds up what all the drivers contribute. So when several drivers share a net (an offset source, a noise injector, a second bridge), the nettype combines them, with no hand-edited testbench code.

  The environment instantiates the model itself, not a copy behind a wrapper, so the checks cover exactly what ships. The environment's own structure is IEEE 1800 throughout: the nettype, the resolution function and the `interconnect` ports. The model it drives has `wreal` ports, though, so running it does need a simulator that supports `wreal`.

  Bridge and wrapper ports are `interconnect` (§6.6.8), so they take on the testbench's nettype instead of naming one. And the scoreboard's golden values are SPICE measurements of your circuit.

- **Equivalence evidence.** The circuit and the model get identical inputs. Their outputs are then compared in the way that matters for that block: waveform error for blocks on the signal path, edge position for blocks on the clock path. The aligned waveforms are saved, so any verdict can be worked out again from its own run directory.

An optional plain-language specification supplies what no netlist can: the block's intended operating range, and the criteria it has to meet (Section 10).

3 Case study 1: two-pole filter

The first case is a two-pole RC low-pass filter. An ordinary block: it takes a signal in and shapes its frequency response on the way out. Here is the whole input netlist, minus its comment lines:

```
2-pole passive RC low-pass filter -- fast-running RNM test case
VIN vin 0 DC 0.9 AC 1
R1  vin  mid  1k
C1  mid  0    15.9p
R2  mid  vout 1k
C2  vout 0    15.9p
.end
```

That is everything the tool is given. No directives naming the ports, no hint about what the block is for, no declared corner frequency. The generated model is then checked in four independent ways:

Chirp (swept-tone) equivalence	0.0290 against a 0.15 threshold	PASS
PRBS (independent stimulus)	0.0097 normalised	PASS
UVM-MS, DC section	49 checks, 0 failed	
UVM-MS, AC section	15 checks, 0 failed	
worst magnitude	0.0674 dB of a 0.25 dB tolerance	
worst phase	2.70° of an 8.00° tolerance	

Here is the head of the generated file, `rc_lpf2_rnm.sv`, exactly as published:

```
// SystemVerilog Real Number Model, auto-generated by spice2rnm.
// Fitted transfer function: H(s) = 1 * 1  / ((1+s/2.4023e+07) * (1+s/1.64656e
    +08))
// poles (Hz): [3823382.3097665044, 26205852.219306976]
```

```
// fit quality: rms_error_db=1.417e-08  max_error_db=5.81e-08

module rc_lpf2_rnm(input wreal in_val, output wreal out_val);
```

The poles were measured from the circuit. Nobody declared them. The file also records how closely its fitted transfer function matches those measurements: the `fit quality` line gives the rms (root mean square) and worst-case error in dB. So you can judge the model's accuracy without running anything.

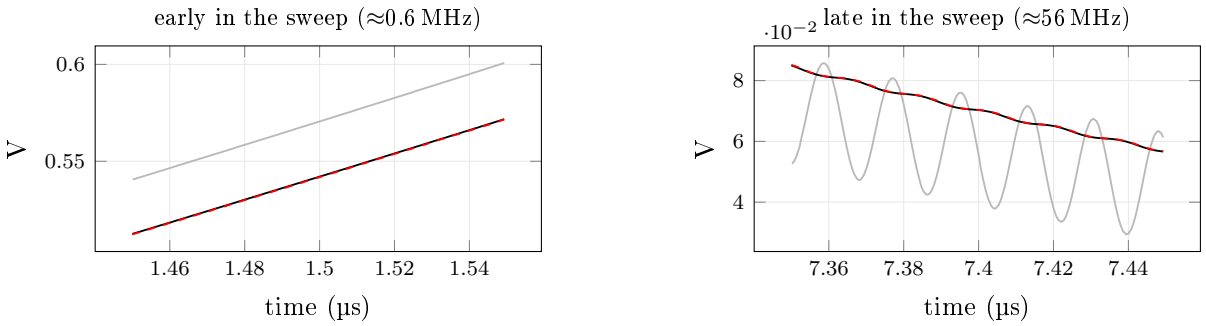
The model also says where its results stop counting as evidence. Every model carries the input range it was measured over. If an input falls outside that range, the model says so instead of quietly returning a value:

```
localparam real DOMAIN_LO = 0.0;
localparam real DOMAIN_HI = 1.8;

a_input_domain: assert (in_val >= DOMAIN_LO - DOMAIN_EPS
                     && in_val <= DOMAIN_HI + DOMAIN_EPS)
else ... $warning("%m: input %0.6g V is outside the characterized
  range %0.6g..%0.6g V. static_curve clamps there, so this model
  keeps running and returns the endpoint value -- it is not
  extrapolating, and it is no longer evidence. Reported once.");
```

Asked for a voltage it was never measured at, the model neither fails nor extrapolates. It clamps: it keeps handing back the value at the edge of its range. That value looks perfectly plausible, which is what makes this case dangerous and why the warning exists. The warning fires once per model, so the log of a long run stays readable.

The PRBS row matters more than it looks. The model is built from a chirp, a tone whose frequency sweeps upward. It is then tested with a pseudo-random bit sequence it has never seen. A model that had only memorised the stimulus it was built from would fall over here. This one does not.



Grey is the stimulus, black the transistors in SPICE, red dashed the generated model. The two panels are two moments of the same chirp, at the simulator's own time resolution. Early in the sweep the output follows the input. Late in the sweep the input still swings but the output barely moves: that is the filter doing its job. Measured over 100 ns windows, the output-to-input swing ratio is 0.99 in the first window and 0.50 in the second, and the model tracks the transistors through both. Data from `waveform_compare.csv` and the PWL (piecewise-linear) stimulus in that run's own `equivalence/` directory.

4 Case study 2: duty-cycle corrector

A clock should be high for half of every cycle. The fraction it actually spends high is its duty cycle, and a duty-cycle corrector puts that fraction back where it belongs when it has drifted. This one is built around a comparator.

How fast is the block? There are two answers, and which one you get depends on how you ask.

Nudge the input a little and watch the output move. That is a small-signal measurement, and the answer is 3.7×10^8 rad/s. Now swing the input hard and time the output edges directly. The answer is between 1.3×10^{10} and 5.1×10^{10} rad/s, which is 35 to 137 times faster.

The slow answer is not wrong, exactly. It measures a sensitive node inside the comparator, and that node really does respond slowly. But the output is not taken from that node. It comes from a buffer that slams between the supply rails without waiting for it. So the slow number is a true fact about a node the output does not follow, and a model built on it runs tens to hundreds of times too slow.

The fast answer is not a single number either. It changes by a factor of 3.9 across the input range, so a model with one fixed speed cannot follow it.

All of which sets a trap for the usual check. That check compares the model's AC (small-signal frequency) response with the circuit's. Both are measured by nudging, so both report the slow node, and the two agree beautifully. The check passes a model that is a hundred times too slow and never breathes a word about it.

Which is why hand-written models of blocks like this one tend to be the ones people trust least.

4.1 A model that follows how fast the circuit really is

While measuring the circuit, spice2rnm notices that its speed depends on input level. So it builds a model whose speed varies with level, the way the measurements say it should. The model also reproduces the circuit's rise/fall asymmetry: the circuit takes a measured 95 ps longer to switch one way than the other. Fed a clock that is high exactly half the time, the model's output duty matches the circuit's to 0.000 percentage points (pp).

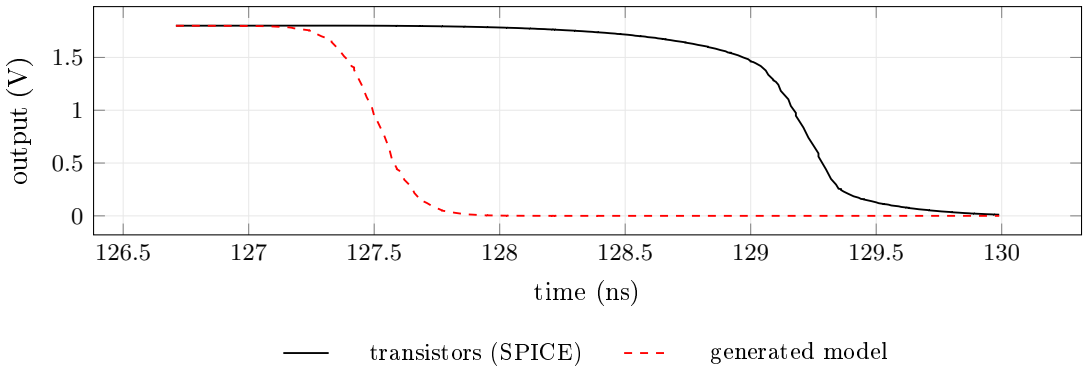
Transient equivalence	0.0769 against a 0.15 threshold	PASS
UVM-MS, DC section	46 checks, 0 failed	
UVM-MS, duty section	14 checks, 0 failed	
worst duty error	0.428 pp of the specification's 1.0 pp	

4.2 Verified by function, not by formula

The UVM-MS environment generated for this block has *no AC section*. That is a decision taken from measurement, not an oversight.

An AC sweep assumes the circuit sits at one operating point and moves only gently around it. A comparator in normal use never does that: it slams between two states. Checked that way, this model fails every single point by more than 55 dB, even though it is demonstrably accurate. That failure says nothing about the model and everything about using the wrong check.

So the generated environment checks what a duty-cycle corrector is actually for: the duty of its output. The tool drives the transistors in SPICE with clocks at two rates and three input duties. Those are the rates and the 35–65% duty range the block's specification states (Section 10). Each time, the tool records the output duty SPICE produces, which becomes the golden value, and the model then has to reproduce those recorded values. The recorded duties span 30 percentage points, wide enough that a model ignoring its input could not pass by luck.



One falling edge, at the simulator's own time resolution. The model puts this edge 1.68 ns early and the next one 1.60 ns early, so the gap between them, which is the duty this block is specified on, survives to 0.03%. An average voltage error over the whole run would call this a large disagreement. The duty check the testbench actually applies does not.

5 Case study 3: Phase interpolator

A phase interpolator does not touch a signal's amplitude. It moves the signal in *time*: each control code nudges the clock edge a little further along. So there is no frequency response worth fitting, and comparing voltages only measures noise. Conventional flows have nothing to offer this block.

spice2rnm recognises what the block does and changes its approach. It measures how far the edge moves at each control code, and builds a delay-line model out of those measurements. Then it generates a *phase* testbench: one check per control code, against the value measured in SPICE. The testbench has no DC or AC section, because neither of those tells you where an edge lands.

Codes measured	9 of 9
Total edge movement, code 0 to 8	−500.1 ps, monotonic
Mean step (1 LSB)	−62.51 ps
Worst step error (DNL)	−100.5 ps (−1.61 LSB)
UVM-MS phase section	9 checks, 0 failed
worst	3.26 ps of a 6.25 ps tolerance

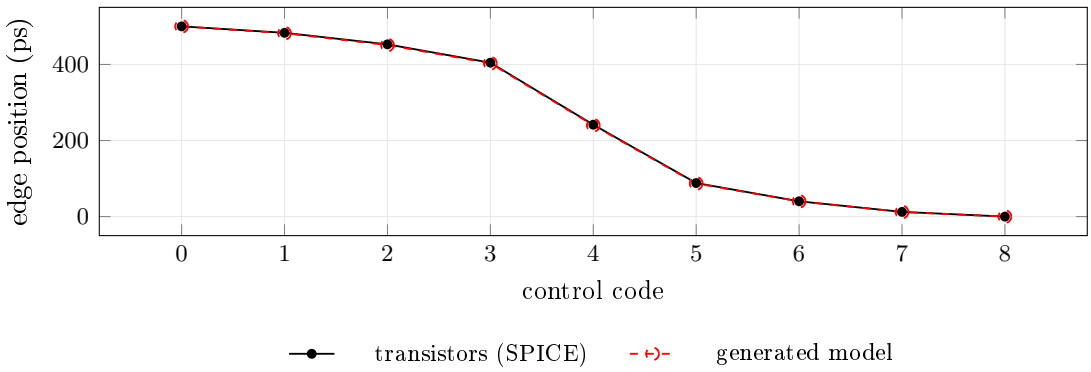
Two terms in that table need a word. **One LSB** (least significant bit) is one step of the control code. The nine codes span 500.1 ps in eight steps, so the average step is 62.51 ps, and that average is the natural unit for everything else here. **DNL** (differential non-linearity) is how far an individual step sits from that average.

The steps in this circuit are very uneven. Subtract each measured edge position from the next and you get eight steps of −17, −30, −48, −163, −153, −48, −28 and −12 ps. Every one of them should be −62.5. The worst is the fourth: it moves 163 ps for a single code change, 100.5 ps more than it should. That is the −1.61 LSB in the table.

This unevenness matters in a real system. A phase interpolator usually sits in a control loop that walks a sampling clock one code at a time to keep it centred in a data eye. That loop assumes each code change is worth about one LSB. Here one code change moves the edge 12 ps and another moves it 163 ps, so the loop's gain varies by more than 13× depending on where it starts.

The tolerance per code is a tenth of an LSB, the finest step the model can resolve. Tight enough to catch a real error, loose enough never to fail on rounding. The uneven steps are the

circuit's own imperfection, reported exactly as measured. The model's job is to reproduce them, and it does. A model with evenly spaced steps would look tidier, but it would hide a real defect from anyone verifying against it.



Edge position at each of the nine control codes: the transistors in SPICE (black) against the generated model (red). There is no waveform to compare here. What this block does is place a clock edge, so edge position is what gets plotted.

6 Case study 4: Phase-locked loop

The first three case studies are single blocks. This one is a *loop*, and that changes what a model has to get right. In a loop a small error does not stay small. It gets fed back round and moves the whole system to a different operating point. So a model that looks reasonable on its own can still park the loop somewhere the real circuit never goes.

The netlist also holds three different measuring problems, and no single method covers all three. The blocks are the netlist's own `.subckt` boundaries:

```
.subckt ro_vco aout cont dd
  ...                          (supplies, and the wires closing the ring)
  xinv1  dd ss sub well cont inm1  inp1  outm1 outp1 gaincell
  ...                                   (7 ring stages)
  xinv11 dd 0  sub well outm1 outm2 inv1
  xinv12 dd 0  sub well outm2 aout  inv2
.ends ro_vco

xcp  pupb pdn dra dd  cpump
xlf  dra vout         lpfilt
xvco aout vout dd     ro_vco
```

The top level has three `x` lines (subcircuit instances), so three models get joined into the runnable whole. `inv1` and `inv2` are instantiated one level down, inside `ro_vco`. They get models of their own and verdicts of their own, which is why the score below is five of five and not three of three. They are not wired into the composed top, because the `ro_vco` model was measured with them already inside it. `gaincell` gets no model at all: it has two inputs and two outputs, so no method that fits one input to one output can measure it, and the composed model does not need one.

Each remaining block is classified from its topology and given a *route*, meaning the measuring method chosen for that block:

Block	Why the default does not fit	Route taken
`ro_vco`	oscillates with its inputs at DC; its output is a frequency, not a gain	tuning curve, scored on rate
`cpump`	a complementary pair driven from independently controlled gates	one $I(V)$ table per control state
`lpfilt`	only R and C, so its state space is exact and needs no fitting	exact state space, read from topology

Not one of the three is a simple input-to-output response. A conventional flow that fitted one to each block would be fitting the wrong quantity three times over.
Appendix A explains what a tuning curve, a control state and a state space are, using this circuit's own numbers, and why an oscillator is compared by its rate rather than by its waveform.

6.1 Checked as a whole system, against the transistors

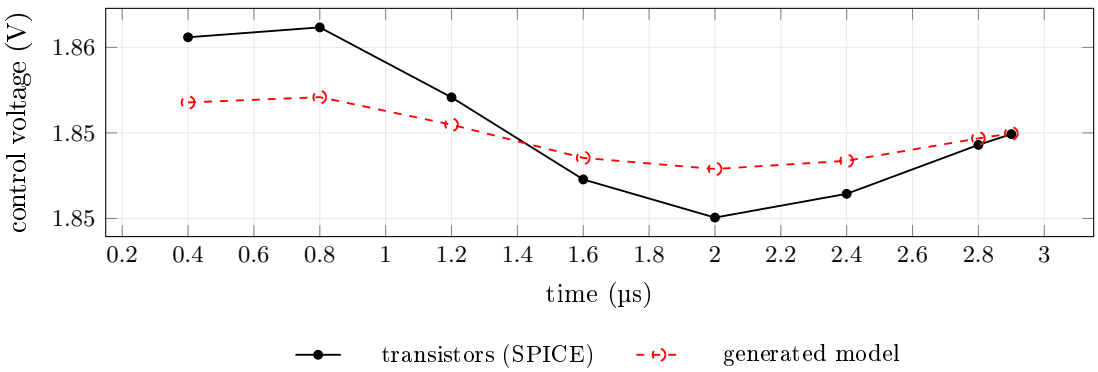
A score for each block tells you about that block and nothing else. A loop asks a different question: wired together, do these models behave like the circuit? An SV–SPICE co-simulation answers it directly. Here that means the digital RTL running in the SystemVerilog simulator while the analog half runs as the real transistors in SPICE (ngspice), with a bridge between them.

The design's own testbench is run twice. In the first run the analog half is the generated models, all in SystemVerilog. In the second it is the transistors in ngspice, co-simulated with the digital RTL. The only thing that changes between the runs is the small piece of code joining the two halves. The thresholds, the expressions and the pass/fail logic are byte-for-byte identical, so the two logs can be compared line by line.

Veylon Systems

Blocks modelled, three routes	3 composed, 2 more generated
Per-block equivalence	5 of 5 pass
Supply noise	20 mV rms, declared in the netlist
System check vs transistor-level circuit	4 boundary signals, 0 failed
`vout` (control voltage)	4.158 mV, 47% of its pass/fail limit
`aout` (VCO output)	0.4% of crossings, limit 2%
`dra` (internal probe)	20.0 mV, 0.9% of its range
Both loops lock at	400.00 MHz
control voltage range, transistors	1.8453–1.8563 V
control voltage range, models	1.8482–1.8522 V
Model half vs transistor half	4.98 s vs 173.6 s
same testbench, same span	35×

One row in that table depends on how busy the machine is, and should be read that way. Between the two most recent runs of this netlist the model half went from 4.37 to 4.98 s while the transistor half went from 81.8 to 173.6 s. The ratio moved from 19× to 35× with nothing about the models changing. A shared machine slows the transistor run far more than the model one, so both halves move together and the ratio follows whichever is more exposed. The speed-up is real, but it is a property of the machine as much as of the models, and four runs in a row have now put it at 16×, 24×, 19× and 35×.
The control voltage is the number to read. The loop keeps its state there, so a model that settles in the wrong place shows it here first. The two runs agree to 4.158 mV and reach the same frequency. They do it with 20 mV rms (root mean square) of random noise on the supply, as the netlist declares. The agreement is looser than it was with a quiet supply, and the supply noise is why: a loop funnels a supply disturbance onto exactly this node.



The control voltage at the boundary between the digital RTL and the analog half: the composed models against the transistors in SPICE, on the design's own testbench, on a node the loop holds near 1.85 V. Both settle to the same place and stay there. There are eight points because the digital side reads this node eight times. The supply carries random noise, so how closely the two curves agree at any one instant is one run's draw, a random sample. The pass/fail limit in the table is what the result is judged against.

The agreement figure changes from run to run, and so does part of the limit. What the agreement itself has to beat does not.

The supply noise is random. Each run makes its own noise from the source the netlist declares, so the model run and the transistor run never see the same noise. The largest difference between them, over any time window, therefore changes from run to run. Seven runs in a row of this same netlist gave 3.61, 1.83, 4.15, 2.05, 1.86, 1.72 and 4.62 mV, then 3.851, then 1.783, then 4.077, then the 4.158 mV quoted above: eleven draws spanning a factor of 2.7. Two of the eleven were run back to back with nothing changed between them. So the figure in the table is one draw from a spread of possible results, not a property of the model: the two most recent runs happen to agree to 2%, while two others differ by more than a factor of two.

The limit it is judged against has two parts, and only one of them is fixed. The agreement itself must come within 0.05 of `vout`'s own scale, in this run and in every other. On top of that sits an allowance for run-to-run variation, and that allowance is *measured*, not declared: it is three standard errors of the difference between two window means, taken from how much the SPICE reference itself moved in this run. So it moves when the noise reaching the node moves. On the nine runs before the last two it came out at 0.128, for a limit of 0.178. On those two the reference was noisier on `vout`: 0.194 and now 0.189, for limits of 0.244 and 0.239. The 4.158 mV works out at 0.112 against that limit, comfortably inside. That ratio is what the verdict rests on, and the record keeps the allowance separately from the 0.05 so a reader can see which part of the limit moved.

The scale has a minimum of its own: 2% of the node's magnitude. Without that minimum the comparison would be unfair here. In the transistor run `vout` moved only 11.1 mV across the window, and measuring a difference against so small a range would report ordinary noise as error.

In general, an agreement figure quoted without its limit tells you less than it looks like it does. A jitter ratio means little without the correlation time it was measured at, for the same reason.

Each node also records which metric its limit applies to. The digital side reads some nodes through a threshold. For those, what matters is when the signal crosses the threshold, not its average value, so they are scored on their crossings. Printing the average beside a crossing limit would show a pass next to numbers that seem to contradict it (0.030 against a 0.020 bar) when the figure actually compared was 0.004.

6.2 The noise the netlist declares is the noise the model gets

The netlist for this case says

```
vdd dd 0 dc {vcc} trnoise(0.02 1e-10 0 0)
```

That is a supply with random noise on it, set by the `trnoise` (transient noise) settings. The first two numbers are the white-noise ones: an amplitude of 20 mV rms, and a time step of 100 ps between new random values. The next two, both zero here, would add flicker (1/f) noise.

A composed model that reads that line and then generates `assign dd_v = 3.3;` has deleted a disturbance the design asked for. Every later check then agrees about a circuit nobody is shipping. What the tool generates instead, in `netlist_top.sv`:

```
int unsigned dd_rng;
initial dd_rng = DD_SEED;
real dd_nw;   // white, sample-and-hold at the deck's NT
initial begin
  dd_nw = 0.0;
  forever begin
    #(DD_NOISE_DT_S * 1.0e15);
    dd_nw = DD_NOISE_RMS * __gauss(dd_rng);
  end
end
assign dd_v = ((dd_v_ext > 0.0) ? dd_v_ext : DD_NOM) + dd_nw;
```

The noise amplitude and the time step are the ones the netlist declares. The supply can still be overridden from outside. The random generator starts from a fixed seed (`DD_SEED`), so a rerun repeats exactly. In a transient simulation of this netlist, that noise source causes 11.80 ps rms of period jitter at the oscillator, ±5% over the two hundred cycles it was measured on. That jitter figure is itself one random draw. A constant supply produces no jitter at all.

The jitter figure is measured for each design, never quoted from somewhere else. Measuring it takes one step first. To measure a block, the tool pulls it out of the netlist, and pulling it out rebuilds every supply as a plain DC source, without the declared noise. So the tool puts the declared noise back onto the block's own supply. Without that step the measurement would describe a quieter circuit than the one being modelled. The generated file records which source the number came from.

The declared source is then carried into the composed model: a new random value is taken at each of the source's own time steps and held until the next. That reproduces the two properties that decide how much jitter a noise source causes, its rms and its correlation time. The noise spectra of model and SPICE agree at low frequency. They part company near $1/NT$ (NT is the time step), because ngspice interpolates between its random values while the model holds them flat.

All seven fields of a `trnoise` declaration are read, not just the two white-noise ones. Where a netlist declares flicker (1/f) noise, the composed model carries it as a Voss–McCartney summation (a sum of random sources updated at octave-spaced rates) across the octaves the run spans. Where it declares random-telegraph noise (a signal that jumps between two levels), the model carries it as a two-state Markov process with exponentially distributed dwell times, using the capture and emission times given. The Voss construction produces $\alpha = 1$. The generated file records both that and the exponent the netlist asked for, rather than quietly swapping one for the other. This PLL's netlist declares white noise only, which is why every figure here shows white noise. Flicker noise dominates close-in phase noise, and a netlist that declares it gets it in the model.

Carrying the noise changes what a comparison means. The model run and the transistor run now each make their own random noise. Two runs of the same circuit therefore differ by chance alone, and with a fixed tolerance the verdict would depend on how long the run was. So each

node's limit is widened to allow for that run-to-run variation, by three standard errors of the difference between two window means. The standard error comes from how much the SPICE reference itself varies, not from the declared amplitude, because how much noise reaches a node is a property of the circuit. On `vout` that allowance was 0.189 of the 0.239 limit in this run. The record carries both, so a reader can see how much of the limit was earned by agreement and how much was allowance for noise, and that the allowance is the part that moves between runs.

6.3 Where the block works is measured, not assumed

The two output-buffer inverters sit in the clock path. Their job is to place clock edges in time, and an average voltage error barely notices that. Both pass an average-error comparison comfortably, yet their error at the single worst instant is 99.9% of the full voltage swing. Those two facts do not conflict. If the model's edge lands slightly late, then for that brief moment the model sits at one supply rail while the real circuit sits at the other. Averaged over time, the moment disappears. For a clock buffer, that moment is exactly the behaviour being verified.

The tool sorts the two inverters by their measured DC curve. With 62% and 61% of swept points within 5% of a rail, the curve is a switching characteristic, not a smooth response. So they get scored on duty cycle instead.

The clock frequency they are scored at is measured too. Where a block operates is a property of the closed loop, and it exists only once the digital side is attached. The same goes for the operating voltage range, which the golden probe (the SPICE run that measures the expected values) already measures. So the probe measures each net's switching rate alongside its voltage range. The two inverter inputs come back at 399.7 and 399.8 MHz, the 400 MHz this PLL locks to. Scored there, they pass at 0.316 and 0.326 pp (percentage points) of duty cycle, against a 0.5 pp limit. The rms voltage error is reported beside each one, 0.112 and 0.129 against the same 0.15 limit it was judged on before, but it no longer decides the verdict.

The model form they are regenerated in has no delay line, so it cannot represent the measured rise/fall skew: 0.584 and 0.657 pp of duty at that clock. Both blocks measure below their own floor, so the skew partly cancels rather than adding up. That makes the floor an upper bound on what the simpler model form costs, not spare margin. The tool states this before scoring, so a pass is not read as unqualified.

A language model cannot supply that clock frequency. Asked five times about the same inverter with the same measurements, the classifier (the language model that judges design intent) proposed clock rates of 2 MHz, 765 MHz, 300 MHz, 10 GHz and 15 GHz. On the run these figures come from, asked once per stage, it proposed 15 GHz for the first and 13 GHz for the second. The lowest of those eight is two hundred times below where the buffer runs. The highest is thirty-seven times above what the buffer's own model can follow. The measurement returns the same answer every run, and its answers for the two stages agree to 0.04%, which no classifier run did. The rule is: design intent from the language model, physical values from the circuit. The clock rate looked like intent, but it is physics.

On what it is actually asked for, the classifier has been consistent: buffer, clock stimulus and duty-cycle metric, on every run of both stages. That is a judgement it can check against a measurement. What varied was the one number it had no way to know.

The pass/fail result names the metric it came from. Asking for the classifier does not guarantee it was used. Every duty-cycle figure above, and the case study's whole table, comes from a run with the classifier reachable and consulted. The same demonstration was also run deliberately with it *unreachable*, to see what a reader would be shown in that case. (Which credential makes it reachable is a property of the build, not of this result: the demonstrations ask the toolchain whether it can call a model, rather than testing for a named environment variable.)

That comparison is worth stating in numbers, because the difference is not cosmetic. With the classifier unreachable, the same two blocks are driven by the default chirp and judged on rms voltage, where they pass at 0.026 and 0.025 against a 0.15 limit. With it reachable they are driven by a clock at the rate the golden probe measured and judged on duty, where they pass at 0.316 and 0.326 pp against a 0.5 pp limit, and the rms figure on that stimulus is 0.112 and 0.129. Three of the five blocks are unaffected either way, because their routes involve no choice of stimulus. The default chirp stimulus is kept, and that is a green result from a measurement that cannot see the property in question, namely edge timing. The two transcripts are otherwise alike and the command line is identical. So the run records which figure each verdict was based on, and the demonstration reports it from that record. Asking for a check on the command line does not prove that it ran.

6.4 The same loop, in SystemVerilog alone

Everything earlier in this case study leans on an SV–SPICE co-simulation set-up already existing: the digital RTL in the SystemVerilog simulator, the analog half as transistors in ngspice, and a bridge joining them. Two steps need it. The golden probe is a run of the design's own testbench in that co-simulation, against the transistors, and it measures the range each block really operates over. The system check runs that same testbench twice. Both need a testbench spanning the two halves, a DPI bridge, and a shared `libngspice` for the bridge to load.

Most designs have none of that. What they have is an analog netlist, some RTL, a specification in English, and nothing wiring the three together. This section runs the same PLL that way: SystemVerilog only, RTL plus generated models, no SPICE anywhere. Call it the SV-only run, to keep it apart from the co-simulation run earlier. In the SV-only run, two things break.

Nothing measures the operating band. A block's operating band is the range of voltages it actually sees in use. That range is a property of the closed loop, and with the loop open there is no loop to measure. Worse, two of the ring oscillator's internal nets are driven by nothing at all, so they have no operating point either.

So the band comes from the two artifacts that do know it, and the run says which one, for every net. Most of it is sitting in the *deck* and is simply read off: a source holding a node at one voltage is that node's band, and a node with a pull-down to ground and a pull-up to the rail cannot leave them. Two numbers are neither in the netlist nor derivable from it: where a *locked* loop keeps its control voltage, and how far the supply is allowed to move. Those two are *declared*, from the operating-conditions table a designer writes anyway. Seven messages cover twenty-one nets, five of them facts about the circuit and two of them design intent:

```
band for 'aout'       DERIVED   as 0..3.3 V, the range its own drivers
                                can reach -- not where it sits; the same
                                for 14 more net(s): xvco.outm1 ...
                                xvco.outm7, xvco.outp1 ... xvco.outp7
band for 'xcp.ss'     DERIVED   as 0 V, fixed: source vss ties it to 0 V
band for 'xvco.ss'    DERIVED   as 0 V, fixed: source vss ties it to 0 V
band for 'xvco.sub'   DERIVED   as 0 V, fixed: source ve ties it to 0 V
band for 'xvco.well'  DERIVED   as 3.3 V, fixed: source vpe ties it to 3.3 V
band for 'dd'         DECLARED  as 3.13..3.47 V from the specification's
                                'Supply (VDD)' row -- not measured
band for 'vout'       DECLARED  as 1.75..1.95 V from the specification's
                                'Control voltage' row -- not measured,
                                because no golden run was available to
                                measure it
```

Nothing on the command line says any of this, and nothing in the specification names a net. The document says "control voltage". The netlist says `cont`. The instance card says `cont` is the top level's `vout`. What ties the three together is the measurement that found which port tunes

the oscillator, so the binding happens after classification and is reported with the block and port it came from. These five numbers used to be five `--band` flags on the command line, which is the one place a number has no owner: it cannot be traced back to the document that justifies it, it is not reviewed with the design, and a copy of it in a script drifts away from its source without a sound.

One more distinction the run keeps. A band read off a pair of rails is what a net *can reach*, not where it sits. So it is the weakest of the three, and it is never narrower than the truth. The charge pump's output can reach either rail too, and in a locked loop it sits at the control voltage instead. That topology is recognised and left without a band, rather than given one that is true and useless.

A band is an input, not a claim. The operating-conditions table is parsed as plain text, with no language model involved, while the rest of the document is read by one. The asymmetry is deliberate. Every other fact taken from a specification gets checked afterwards against what the circuit was measured to do, and the measurement wins. A band cannot be checked that way, because it decides the range the circuit is measured over in the first place. A number with that much authority must not depend on whether a model happened to be reachable, so it is read by the part of the toolchain that always runs.

The tool also reads the specification itself. The demonstration hands it the specification document along with the netlist and the RTL. Here is what it took from that document, in the run these figures come from:

```
read as: Document states "PLL -- 400 MHz ring-oscillator
  phase-locked loop" with "seven-stage differential ring" whose
  "behaviour is a frequency, not a gain" -> oscillator/vco
  environment; "Output ... 400 MHz" is the highest stated
  frequency; "Reference ... 10 MHz" is the lowest stated rate;
  "20 mV rms of wideband noise is specified on it"; "hold 400 MHz
  to within 2%" and "No small-signal specification ... Do not
  verify any of them against a frequency response" implies
  timing-based verdict; control-voltage swing range quoted as
  "characterizable from 0.5 V to 2.8 V".
```

These are facts about what the part is FOR, which is a different kind of statement from the operating band above. They choose the stimulus and the verdict, and they are claims the measurement can contradict. Section 10 gives the rule that governs them: every fact taken from the document is checked against what the circuit was measured to do, and the measurement wins. Here nothing disagreed.

More useful is what it refused to let pass. The document names 20 mV rms (root mean square) of supply noise as a condition the part has to work under. The reader placed that correctly: an operating condition belongs in the stimulus, not in the model. It then reported that the generated environment does not drive that noise:

```
spec: the document states 20 mV rms of supply noise as a condition
  to work under. That is a property of this block's environment, so
  it belongs in the stimulus rather than in the model
NOTE: the supply-noise condition is not driven by the generated
  environment yet; the composition carries only the noise the
  netlist itself declares
```

So a requirement the tool cannot yet meet gets flagged in the output instead of quietly dropped. That is the behaviour worth having from a language model in a verification flow: not a guess at a fix, but a gap it can name.

It did not supply the five bands. A person still reads those off the specification's table and types them on the command line. The run prints the phrase `a spec DECLARES` for them.

That phrase is the band mechanism labelling a figure a human supplied. It does not mean the reader extracted it.

This passage costs the paper some reproducibility. Every other figure in the paper regenerates identically from its run. The quoted text above does not, because a language model picks different words each time. What stays stable is everything around it: the bands, the measurements and the verdicts. All of those were byte-for-byte identical across this run, across two earlier runs where the model read the document differently, and across the run before those, where the reader did not run at all because no model was reachable. What a model changed was what was *read*, never what was measured.

Neither a derived band nor a declared one is a measured band, and the run never dresses either up as one. Every message that uses a band names its source in those words: `a spec DECLARES as 1.7500..1.9500 V, not measured` for the control window, and `the deck itself gives as 0.0000..3.3000 V` (a complementary pair drives it between the rails) `derived and not measured` for a net inside the ring. The blocks the open loop cannot reach say so too, instead of quietly measuring something else.

Per-block models are still not a system. Checking each block on its own does not show that the loop works. Without a co-simulation, that question still needs an answer, so the SV-only run closes the loop in *pure SystemVerilog*. It uses the composed models and the design's own `pfd.sv` and `divn.sv`, instantiated unchanged, in one simulator. No DPI import, no bridge, no SPICE process anywhere in it.

The user states no *topology*. An earlier version of this demonstration shipped a JSON file naming which RTL port landed on which net. That file is gone. The wiring is followed out from the one clock the analog half produces, by port *direction*, and not one port name is read on the way.

That last part is the point, because on this circuit a port name is actively misleading. In ngspice's own PLL example the detector is instantiated `Xfpdet d_divout d_ref` against a subcircuit declared `.subckt f-p-det d_R d_V`. So `d_R` carries the *divided* clock, the opposite of what it reads like. A loop wired from names is confidently backwards there. Here is what the run worked out, and what from:

```
pfd  . ref_clk -> clk_out       divn . clk_in  -> aout_clk
pfd  . div_clk -> ref_clk       divn . clk_out -> clk_out
pfd  . dn      -> dn            analog aout    -> aout_clk
pfd  . up      -> up

the oscillator's control net = vout
    the port block_classify measured as its control source
the net that becomes a clock = aout
    block 'ro_vco' was measured to oscillate on its port 'aout'
the divide ratio = 40
    parameter N of module divn defaults to 40 in its own header
the reference rate = 1.00014e+07 Hz
    the deck holds 'vout' at 1.8495 V, where the measured tuning
    curve puts block 'ro_vco' at 400.056 MHz, and the ratio is 40
```

One fact about this loop is in neither the netlist nor the measurements. Which of the detector's two outputs speeds the oscillator up is a property of the RTL's *body*, and both wirings elaborate and run. This used to be settled by building both loops and keeping whichever one locked. That works, costs a second loop run every time, and leaves the question answered by an experiment rather than by the document that ought to state it. The specification states it instead, in one line that names a port of the customer's RTL and the control voltage, and no gate of the charge pump:

> **The detector's `up` output raises the control voltage.**

That line is read as text, with no model and no credentials, for the same reason the operating-conditions table is: it decides how the loop gets *wired*, so it must not move with whether a model could be called.

And "raises the control voltage" is not "speeds it up". This ring's tuning curve falls, so the output that raises the control voltage is the *slower* one. The run does that conversion with the sign it measured, rather than assuming the two agree. Reading one as the other is exactly the mistake the statement exists to remove, and on this circuit they point opposite ways.

The stated sense does not outrank the circuit. The other pairing is still written out beside the one that was built, and if the stated one fails to lock, it is built and run too. A loop that locks is a measurement and a document is not, so the measurement wins and the run reports the document as wrong about its own detector. On this deck the stated pairing locked first time, so one loop was built and the alternative stayed a record.

The user does *not* state the polarity of either charge-pump gate, meaning whether it is active high or active low. The tool works that out from two things the measuring has already established: the sign of the oscillator's tuning curve, and the direction of the current each control state drives. The generated file records the numbers it came from:

```
pdn <- dn, active HIGH (derived)
  ro_vco measured at 15 control points, 0.495 to 2.805 V, over which
    its frequency FALLS 804.47 -> 16.74 MHz: Kvco is NEGATIVE
  lpfilt: current into dra reaches vout through a state space whose
    input vector is non-negative throughout, so positive current
    RAISES it
  with cpump quiescent (pupb=1), pdn at 1 drives -0.00447 A into dra
    and at 0 drives -1.961e-09 A, both at 1.849 V -- active HIGH
pupb <- up, active LOW (derived)
  ... the same two measurements, and with pdn=0, pupb at 1 drives
    -1.961e-09 A into dra -- so its asserted level is the LOW one
```

On this one pump, `pupb` comes out active *low* and `pdn` active *high*. A port name does not reliably carry that distinction, and charge pumps wired from their names have shipped upside down before.

The table below summarises the SV-only run. "Route" here means the measuring method the tool chose for a block.

SV–SPICE co-simulation, at any stage	none
Models generated, using four routes	5
Operating bands	5 derived, 2 declared, 0 measured
Per-block equivalence	5 of 5 pass
Loop closed in	SystemVerilog only
Loop sense, stated by the specification	1 line, no net named
Gate polarities derived from measurement	2 of 2
Gate pairings built and run	1 of 2, the stated one
Divider output vs the reference it follows	10.001348 vs 10.001401 MHz
against a 2% pass/fail limit	−0.0005%, locked
Control voltage across the window	1.8488–1.8506 V
drift end to end	−0.26 mV
Environments generated	3 per block, 1 over the loop
blocks refused, with a reason	2
run, and reporting results	4 of 4
Characterisation, loop included	11 min
The four environments, run afterwards	11 min

A UVM-MS environment over the whole loop. The loop above is a plain testbench: it drives a reference clock and checks a frequency. The SV-only run also generates a full UVM-MS environment around it, with a sequence, sequencer, driver, monitor, agent and scoreboard. It also has an MS proxy split in two: an abstract half that the components talk to, and a concrete half in the top level. Because of that split, neither the driver nor the monitor holds a hierarchical path into the design.

It is the only *system-level* environment this tool generates with all of those components. The reason is the stimulus:

environment	seq	drv	sqr	mon	agt	prx	scoreboard
block, transfer function	•	•	•	•	•	•	`uvm_scoreboard`
block, oscillator	•	•	•	•	•	•	`uvm_scoreboard`
system, SV–SPICE co-simulation	—	—	—	•	•	•	`uvm_scoreboard`
system, SV only	•	•	•	•	•	•	`uvm_scoreboard`

The system environment for the co-simulation run has the observing half and none of the driving half. The testbench it wraps brings its own stimulus, so there is nothing for a sequence to ask for. The SV-only run has no such testbench. Its stimulus is the reference clock, and this environment generates that clock, so a sequence, a sequencer and a driver all have real work to do.

The oscillator's block environment uses the same pattern, and the pattern is what makes both of them extensible. The timing measurement lives in the proxy's concrete half in `top`, the only scope that can see the design's signals. The driver, monitor, agent, scoreboard and test hold no references to either signal, against eight in `top`. A second observer can subscribe to the monitor's analysis port without restructuring anything. And *one* implementation of "time a clock" serves both this environment and the loop one.

The reference rate itself is not a round number any more. It is the measured tuning curve read at the control voltage the deck states, divided by the ratio the divider's own header declares: 1.8495 V puts the ring at 400.056 MHz, and 40 of those make 10.001402 MHz. The loop environment's sequence and driver use that timing to step the reference frequency up by 0.5% and back down again. That is why the results below have three measurement windows instead of one settled snapshot. A model frozen at its initial condition would produce a single settled snapshot too, so one snapshot proves little.

Its expected values come from two places, and every line of its report says which. Two come from the specification: reach the reference frequency within tolerance, and keep the control voltage inside the stated window. Two come from this run's own measurements, and the second of those is the valuable one. The oscillator's tuning curve was measured point by point in ngspice before any of this existed. Invert that curve at the frequency the loop is asked for and you get the control voltage the oscillator needs to produce it.

Window	divider output	control voltage
nominal, 10.001402 MHz	10.001342	1.8488–1.8505 V
stepped, 10.051409 MHz	10.050600	1.8435–1.8457 V
returned, 10.001402 MHz	10.000518	1.8490–1.8506 V
rate, against a 2% bar	3 checks, 0 failed, worst 0.0088%	[SPEC]
control window, 1.75–1.95 V	3 checks, 0 failed	[SPEC]
against the measured curve	3 checks, 0 failed, worst 0.89 mV	[MEASURED]
boundary finite	3 checks, 0 failed	[MEASURED]

The stepped window is the one to read. The reference went *up* and the control voltage went *down*, which is what a negative K_{vco} requires. It also went down by the amount the measured

curve predicts. From its 15 points, that curve puts 400 MHz at 1.8496 V, with ±2% spanning 1.8291–1.8705 V. The loop settles 0.89 mV from the prediction, inside a window 41.5 mV wide.

That check is worth reporting because it can fail. A loop can reach the right frequency at the wrong control voltage: through a charge pump with the wrong sign, a wrongly fitted slope, or a divider ratio the reference happens to absorb. Counting clock edges cannot see any of that. Shift the measured table by 100 mV and exactly those three checks fail, by about that same 100 mV, while the two specification checks still pass. An environment built only from the specification would have passed the same run.

Three per-block environments are generated beside it: one for each of the two clock buffers, and one for the oscillator. Two blocks are refused, with the reason written out. The charge pump's result is a current for each control state, and the filter's is a voltage produced by an injected current. Every generated environment scores a voltage against a stimulus, so neither of those fits.

All four are then run, which is a stronger claim than all four being generated. A file count shows that a generator produced something. It says nothing about whether the output elaborates, compiles against the Accellera UVM library, or scores anything. So each environment is executed and its scoreboard result reported:

Environment	what its scoreboard concluded	time
`inv1`	54 DC checks, 0 failed, worst 0.001 mV of a 6.6 mV bar	6 s
`inv2`	54 DC checks, 0 failed	111 s
`ro_vco`	15 breakpoint + 5 held-out, 0 failed, worst 0.024%	219 s
the loop	12 checks over 3 windows, 0 failed	336 s

The two inverter results are the weakest of these, and the last paragraph of this subsection says why. All four environments are published, and all four were run from the published copy rather than from the run directory, so what is published is exactly what ran.

What this result does not carry. There are two limits here, and neither one is small.

First, there are no transistors in this loop, so nothing in the SV-only run is evidence about the circuit itself. A loop that settles at a plausible but wrong point still settles, and only the co-simulation run earlier can catch that. The verdict covers this modelled loop, at this reference frequency, starting from the initial condition the netlist states.

Second, the SV-only run's 5 of 5 is not the same 5 of 5 as the co-simulation run's. A clock buffer's verdict only means anything with the duty-cycle metric, and that metric needs the clock's frequency. The golden probe is what measures that frequency, and the SV-only run has no golden probe. So `inv1` and `inv2` fall back to comparing average voltage: 0.021 and 0.026 against a 0.15 bar. Both pass, but that check is blind to where the clock edges fall, for exactly the reason Section 6.3 gives. The two runs agree on the count and differ on what the count is worth.

7 Results at a glance

Case study	Environment	Checks	Failed
Filter	DC + AC	49 + 15	0
Duty corrector	DC + duty	46 + 14	0
Phase interpolator	phase per code	9	0
Phase-locked loop	SV–SPICE co-simulation vs SPICE	5 + 4	0
the same loop, SV only	UVM-MS loop env, derived/declared bands	5 + 12	0

The first three case studies run end to end in under 16 minutes on a laptop. That covers measuring the circuits, generating the models, both simulators, and every generated environment. The PLL adds about 11 minutes more, most of it spent measuring the ring oscillator's tuning curve (its frequency against control voltage).

The SV-only variant takes its time for the same reason. On the run these figures come from, the pipeline took 11 minutes with the loop closing included, and running the four generated environments afterwards took another 11. The total moves between runs, because the oscillator's refinement passes vary and because the machine is shared.

Its twelve extra checks belong to the loop environment and cover three windows. The rate and control-window limits come from the specification. Agreement with the measured tuning curve, and finiteness (no NaN or infinite values), come from this run's own measurements. Its 5 per-block passes carry less weight than the 5 in the row above, for the reason given in Section 6.4.

Each case is one command. `demo_4_pll.sh` is the PLL checked against the transistors: the digital RTL in the SystemVerilog simulator, the analog half as transistors in ngspice, a bridge joining them. `demo_4b_pll_nocosim.sh` is the SV-only row beneath it.

The co-simulation case runs the same way as the others, but it needs more installed. Being the analog half of a co-simulation, it needs the other half: the RTL, the DPI bridge, and a *shared* libngspice. That last one is a different ngspice build from the standalone binary the first three cases use.

The script looks for each of those pieces before it does any work. If one is missing it stops there and says which piece and where to get it, rather than failing twelve minutes into a run.

spice2rnm's outputs are plain IEEE 1800 SystemVerilog, tied to no simulator vendor. So that anyone can reproduce what is here, all of it was produced with open-source simulators. **ngspice** did the measuring and the reference transient runs. **xezim**, an open-source IEEE 1800 simulator that runs the Accellera UVM library[1], ran both the SV–SPICE equivalence co-simulation and the generated UVM-MS environments. No commercial licence was used anywhere in this document.

The generated files are published so you can read them, exactly as the tool wrote them: the models, the complete environments, and the evidence files. They are at

`https://github.com/vrajeshprakhya/spice2rnm-demos`

under `showcase/`, stamped with the revisions of the tool, the demos and the simulator that made them. The repository holds:

```
spice2rnm-demos/
  README.md           what this repository is, and is not
  .gitignore          run scratch; the showcase is curated, not
                      whatever a run left behind
  demo_1_lpf.sh  demo_2_dcc.sh  demo_3_pi.sh
  demo_4_pll.sh  demo_4b_pll_nocosim.sh  run_all.sh
                      the five case studies, each one command;
                      run_all.sh chains the first four
  netlists/           the four SPICE inputs, exactly as the demos
                      run them: rc_lpf2, dcc, pi_therm, and the PLL
                      netlist with the supply-noise card it runs under
  specs/              the three specifications in English: the
                      filter's, the duty-cycle corrector's (demo 2
                      reads it) and the PLL's (demo 4b reads it)
  harness/            9 files: 8 measurement scripts the demos
                      call, plus the JSON stating how the PLL's
                      loop is wired
  showcase/           the generated artifacts, byte-for-byte
    lpf2/             the baseline case, 13 files + equivalence/:
      rc_lpf2_rnm.sv          the model, wreal ports
```

[1] `https://github.com/vrajeshprakhya/xezim`

```
    rc_lpf2_ms_types_pkg.sv nettype real rc_lpf2_wire,
                            resolved by summing its drivers
    rc_lpf2_dms.sv          interconnect ports, straight through
    rc_lpf2_bridge*.sv      MS bridge and its driving core
    rc_lpf2_proxy_pkg.sv    agent <-> bridge API
    rc_lpf2_ms_pkg.sv       item, sequence, driver, monitor,
                            agent, and the SPICE goldens
    rc_lpf2_ms_scoreboard.svh  the checks and tolerances
    rc_lpf2_ms_tb/test.svh  environment and test
    top_rc_lpf2_ms.sv       nets, DUT, bridge, test
    run_rc_lpf2_ms.sh       compiles and runs it
    result.json             fit, verdict, warnings
    equivalence/            what the check actually ran and found
  dcc2/             same shape, 14 files + equivalence/: no AC
                    section, adds duty goldens in dcc_ms_pkg.sv
                    and the run log pipeline.txt
  pi/               same shape, 13 files + equivalence/: an MS
                    bridge, proxy and agent like the others, with
                    phase-per-code goldens and no DC or AC section
  pll_cosim/        53 files + 9 under equivalence/. Case study 4,
                    checked against the transistors:
    11 at the top   five block models, three interfaces, the
                    composed analog top, the node package,
                    result.json
    env/inv1/  12   and env/inv2/ 12 -- a full UVM-MS environment
    env/inv2/  12   each: item, sequence, sequencer, driver,
                    monitor, agent, scoreboard, env, test, MS
                    proxy and bridge
    env/ro_vco/ 10  the oscillator's, same components, goldens a
                    measured tuning curve rather than a DC sweep
    env/cosim/  8   the SYSTEM environment: the design's own RTL
                    and every block's model elaborate together,
                    goldens the transistor run's own statistics,
                    carrying a period-jitter monitor at 1 fs
  pll_nocosim/      58 files + 9 under equivalence/. The same
                    circuit run in SystemVerilog only:
    11 at the top   the same models, interfaces and composed top
    env/inv1/  12   env/inv2/ 12, env/ro_vco/ 10 -- as above
    env/rtl_loop/ 7 the LOOP environment. The only SYSTEM
                    environment generated with the stimulus half,
                    because it owns its stimulus: proxy package,
                    package, scoreboard, env, test, top, run script
    loop/       2   the plain harness it replaces: one SystemVerilog
                    file and the script that runs it
    rtl/        4   the customer's pfd.sv and divn.sv, the English
                    specification, and the loop's topology in JSON
  house_lib/        4 files, NOT generated: ng_ams_pkg.sv
                    declaring nettype real ng_anet, and two
                    hand-written models using it
  lpf2_house/       13 files: lpf2 again, joined to house_lib --
                    no types package, and rc_lpf2_rnm_ng_anet.sv
                    in place of the wreal boundary
  STAMP             per case: when it was curated, from which run,
                    at which tool and simulator revisions; and the
                    last full run's summary
  refresh.sh        regenerates everything; the only way this
                    directory is ever updated
whitepaper/         this document, source and PDF
```

7.1 What the equivalence folders hold

Every case study publishes an `equivalence/` directory. It holds what the check ran, what it found, and a list (`MANIFEST.txt`) of the large files kept out of the repository.

Case	Scored on	Published verdict
Filter	rms voltage	0.0290 against 0.15
Duty corrector	rms voltage	0.0769 against 0.15
Phase interpolator	phase per code	worst 1.832 ps at code 2, tolerance 6.251 ps
Phase-locked loop	five routes, then the whole system	five block verdicts, and the system over 116,018 samples

In that table, "rms" is root mean square: the rms voltage error between model and SPICE, against a pass/fail limit of 0.15. A "route" is the measuring method chosen for a block, and the PLL has five blocks, so five routes.

Start with two files. `equivalence/verdict.txt` sums up the check and its conclusion, one line per route: the metric, the worst point, the pass/fail limit, and the verdict. Beside it, the case's `result.json` is the full record that line was taken from. It records what was driven, what was compared, over how many points, and every warning raised along the way.

Each folder carries three things. First, `verdict.txt`, which the tool copies out of that run's own `result.json` rather than retyping, so a number here cannot disagree with the run that produced it. Second, the simulator and SPICE logs, which show what actually ran. Third, a comparison table with one row per measured point. The phase interpolator's `phase_compare.csv`, for instance, has nine rows, one per control code: the measured delay, the model's delay, and the error. The PLL's `rc_compare.csv` is the loop filter's response to an injected current, sample by sample.

Why some files are listed in `MANIFEST.txt` instead of published. Some output files are large, easy to regenerate, and add little for a reader, so they are not committed. Git keeps every committed file in the repository's history forever, so anyone who clones it downloads them, even after they have been deleted. Instead, `MANIFEST.txt` names each such file with its exact size and the command that recreates it. You can see what exists and how big it is, without every future clone having to carry it.

The large files all come from the waveform routes. Those write one row per simulator timestep, not one per measured point. The filter's aligned comparison is 18.6 MB and its generated testbench 18 MB. The duty corrector's comparison is 2.8 MB. The PLL is the bulk of it: its co-simulation waveform file alone is 202.8 MB, two thirds of everything the five cases produce.

Together that comes to nearly 296 MB, and `refresh.sh` regenerates all of it in half an hour. Committing it would leave the repository permanently heavy for no lasting gain. What is published comes to 1.4 MB. Of that, the SV-only PLL is 546 kB: four environments, the loop harness, the customer's RTL and both specifications.

The rule goes by file size, not by file name. A comparison table small enough to read is evidence. One large enough to be a waveform is a trace. Routes differ in which they produce: a network compared point by point writes a table, while a waveform compared sample by sample writes a trace. So the rule has to look at the file itself, not at what it is called.

The repository leaves two more things out on purpose. One is the spice2rnm tool itself: the product is licensed separately, and this repository is its evidence. The other is the build litter from the runs, meaning scratch files from the measuring, compiled simulator objects and DPI shims. `refresh.sh` filters those out, so the showcase holds what a user would keep, not everything a run leaves behind.

8 What the equivalence check compares

Every model in this paper was checked the same way. The generated SystemVerilog goes into a digital simulator and gets an input signal. The transistors in SPICE get the very same input. The two outputs are then lined up in time and compared point by point. Both traces are saved, so anyone can open the run's own folder and work the number out again.

Two things about this check matter more than the numbers it produces.

The first is that **it tests the file you are actually handed**. Some tools report a weaker number: how well the fitted equations matched the measurements they were fitted to. That number never opens the generated file, so it would look identical even if the file had been written out wrongly. This check loads the generated file into a simulator and runs it, so a code-generation bug makes it fail.

The second is that **each block is judged on its own job**. One error figure for every block would miss the point on most of them. An average voltage error cannot see where a clock edge lands, and an oscillator's frequency has no voltage error to measure in the first place.

If the block	then compare	and score it as
passes a signal through	the output waveform, on a sweep and on a second, unrelated input	the error, against how far the output itself moves
drives a clock	where the edges land, at the rate measured in use	duty error, in percentage points
moves an edge per setting	the crossing time at every setting	picoseconds, plus step size and evenness
oscillates	the output frequency, as the control voltage varies	a percentage of that frequency
switches a current	the current it drives, in each switch position	a percentage, for each position
is passive R/C	the output's response to a current pushed into it	the error, against how far the output moves

The fourth row cannot be scored like the first. Two oscillators at the same frequency still drift apart in phase, so subtracting their waveforms would score a perfect model as completely wrong. Appendix A works this through.

8.1 What keeps a figure from flattering itself

Some measurements are held back. When a block is measured at many points, a share of those points is kept out of the model-building and used only for scoring. So the headline figure is not the fit marking its own homework. The charge pump in case study 4 reports 0.549% error on held-back points. The ring oscillator reports its held-back frequency error directly.

An input the model has never seen. The filter in case study 1 is built from a frequency sweep. It is then tested with a pseudo-random bit sequence it was never shown (0.0097). A model that had merely memorised the data it was built from would be caught there.

The report says what range the answer covers. A comparison proves nothing about conditions it never tested, so the report states the conditions it did test. The ring oscillator is the clearest case, and the honest answer there is two numbers. Inside the control-voltage band the loop actually uses, its held-back frequency error is 0.024%. Out at the far edge of the wider measured range, where the tuning curve bends hardest and the loop never goes, the error reaches 4.987%. Both numbers get printed, and both come from the SV-only run of Section 6.4.

The co-simulated run of the same block, whose band the golden probe measures rather than the specification declaring it, reports 0.003% and 5.348%. The verdict is taken in the band that matters, and the less flattering number is printed rather than quietly dropped.

Known limits of the model shape are priced in advance. Sometimes the chosen model shape simply cannot do something the circuit does. When that happens, the cost is stated up front, in the same units as the verdict. The inverter timing asymmetry in case study 4 is an example, quoted as duty error at the clock rate in use. A pass is then read against that floor, not taken as a clean win.

8.2 Blocks, and then the system

The table above scores blocks one at a time. Wiring blocks together raises a further question: errors that are small in each block need not stay small once everything is connected. Three levels of check answer it.

First level: the netlist alone. The assembled model is driven at the external pins of the design. Its output is compared against the full transistor circuit in SPICE, driven the same way.

Second level: an existing SV–SPICE co-simulation. This level applies when you hand the tool a co-simulation set-up: the digital RTL in the SystemVerilog simulator, the analog half as transistors in ngspice, a bridge joining them. Case study 4 reports this level first. The design's own testbench is run twice. Once with the generated models as the analog half, once with the transistors. Nothing else changes. The digital logic, the limits it checks against and the pass/fail rules are byte-for-byte identical in both runs. Only the small piece of code joining the two halves is swapped out. That makes the two runs comparable signal by signal: 4 signals crossing between the halves, 0 failed, the control voltage agreeing to 4.158 mV on a rail carrying 20 mV rms (root mean square) of declared noise, and both versions locking to the same frequency.

Third level: SystemVerilog only. Most designs have no co-simulation set-up. What they have is an analog netlist, some RTL, and nothing joining the two. Here the models are composed and the customer's own RTL is wired around them in SystemVerilog, with no bridge and no SPICE anywhere in the loop. The question is whether the assembled design does its job. A full UVM-MS environment drives it and checks it against two things: the specification, *and* what this run measured. The oscillator's own tuning curve, for instance, says which control voltage ought to give the target frequency. A loop that hits the right frequency at the wrong control voltage is therefore caught, not counted as a pass. Section 6.4 runs the same PLL this way. It locks to −0.0005% of its reference, with the control voltage 0.89 mV from where the measured tuning curve puts it. This is deliberately a weaker claim than the second level makes, because there are no transistors in the run to disagree with it.

A check that does not apply to a given run is reported as not applicable, with the reason. It is never quietly counted as a pass.

8.3 One thing a score at an operating point cannot see

Every check so far is *static*: it compares a value at a fixed operating point. That suits most of what a model claims. It is blind to one error that matters, though. It cannot see what the model does when the control or supply voltage is *moving*.

An oscillator model has to decide when to place its next edge, and there are two ways to do it. One is to read the control voltage at the instant of the previous edge and schedule a whole half cycle from that single reading: call that the *sampling* form. The other is to add the control voltage up across the cycle as it changes, which is what the real circuit does: call that the *integrating* form.

Both forms use the *same* tuning curve. So a check that measures frequency at fixed control voltages scores them identically, and in practice it passes both. Feed them the same *moving* control voltage and they come apart. On the reference ring oscillator, with a control disturbance

of 100 ps correlation time, the sampling form produced 3.37× the period jitter of the transistors in SPICE.

The size of that gap is not fixed. It depends on how fast the disturbance moves. The predicted ratio is $\sqrt{T_{\text{half}}/\tau}$, where τ is the disturbance's correlation time (roughly, how long it stays at one value) and T_{half} is the half period. For 1.25 ns against 100 ps that predicts 3.54, within 5% of the measured value above. A faster disturbance pushes the two forms further apart, so a ratio quoted without its τ says less than it looks like it does.

So the tool tests the moving case directly. A known disturbance is injected in SPICE at several amplitudes. Then the exact disturbance waveform *that ngspice actually solved* is replayed into the model, sample for sample.

The replaying is the part that matters. Give the model its own separate random noise of the same size, and the check ends up comparing two unrelated random sequences, whose agreement depends more on how long the run was than on the model. Replaying the same waveform tests the thing worth testing: how a disturbance turns into jitter.

Both of the oscillator's inputs are tested, not just the obvious one. On the reference ring, the supply voltage moves the frequency by 450.6 MHz/V: about as much as the control input does, and in the opposite direction. A model with no supply input could not show what a supply droop does at all. The overall verdict is the worse of the two paths.

In the table below, each row gives the rms (root mean square) period jitter of the circuit and of the model, and the model-to-circuit ratio. A ratio of 1 means perfect agreement.

Path	Injected	Circuit	Model	Ratio
either	none (floor)	0.051 ps	0.000 ps	no limit
control	5 mV rms	2.396 ps	2.352 ps	0.981
	20 mV rms	8.743 ps	8.690 ps	0.994
	50 mV rms	26.719 ps	25.618 ps	0.959
supply	5 mV rms	2.808 ps	2.640 ps	0.940
	20 mV rms	11.617 ps	12.139 ps	1.045
	50 mV rms	28.422 ps	27.656 ps	0.973

Each ratio is one random sample, like the boundary figures earlier, not a fixed property of the model. Every injection uses a fresh random noise waveform, and an rms measured over two hundred cycles carries a sampling uncertainty of about 5% either way. That is why the same six points on the previous run read 0.987, 0.946, 0.925 and 0.896, 0.953, 0.970. That spread is why the pass/fail limit is 0.25 rather than something that looks stricter. A tolerance tighter than the measurement's own uncertainty would fail correct models, at a rate set by how long the run happened to be.

The floor row is measured, but reported for information with no limit. With nothing injected at all, the reference still reports about 54 fs of jitter. That figure is the precision with which the edge-finder locates a threshold crossing. It is not anything the circuit did. Asking the model to reproduce it would be asking it to reproduce a measurement artefact. Instead it is the floor every other amplitude has to clear before its result counts.

That agreement was not obtained by fitting anything. Simple physics predicts it. Take the supply path, where the sensitivity is measured separately. If the disturbance held still for a whole cycle, then 450.6 MHz/V on a 2.509 ns period would predict 14.18 ps of period jitter for 5 mV rms. The circuit gives 2.797 ps, a ratio of 0.197. The reason is that the oscillator averages a fast disturbance out over its own period. Averaging about T/τ independent pieces of noise shrinks the effect by $\sqrt{\tau/T} = \sqrt{100.3\,\text{ps}/2509\,\text{ps}} = 0.200$, which is what the circuit shows.

This holds across the whole tenfold range of amplitudes, not just at one point: the ratio is 0.197, 0.223 and 0.203 at the three amplitudes checked. An oscillator that did not average its input would sit at 1, five times higher. All three points land within 12% of a prediction with no fitted parameters, and each carries the same few per cent of sampling uncertainty as the ratios

above. The integrating model reproduces this because it integrates. The sampling model does not.

The tool decides for itself whether this check is worth running. For most designs the honest answer is no, and the check costs a quarter of an hour. Two things can say yes.

The first is the design's own closed-loop behaviour, measured. The tool measures how far the control voltage moves within one oscillator period, converts that into period movement using the tuning curve's slope, and compares it with what the reference measurement can resolve. On this PLL the answer is no, and it comes with numbers. On a run where this was the deciding input, the control moved 12.18 µV rms per 2.503 ns period. That is worth 0.0288 ps against a 0.0521 ps floor, or 0.55× what the reference can resolve. A loop filter holds its control voltage still. That is its whole purpose.

The second is the netlist, and it is what triggered the sweep above. This netlist declares 20 mV rms of noise on the ring's supply, and a design that writes noise into its own netlist is asking for noise to be simulated. The rule matches on the net name, not the source name, because block extraction rebuilds every supply as a plain DC source. The extracted fragment carries `VBIAS2 dd 0 DC 3.3`, where the top level wrote `vdd dd 0 dc {vcc} trnoise(...)`. The net name survives extraction. The source name does not.

There is no flag to force the check, because asking for a check is not evidence that it is needed. Where the tool has no closed-loop evidence either way, it runs the check rather than skipping it. Having no evidence that a check is needed is not the same as deciding it is not.

What the sweep measures, and what it does not. The sweep drives the ring oscillator on its own, on the fragment extracted for measuring it. That is the right scope for its question: does this model turn a disturbance into jitter correctly? It is a different question from what the PLL does with a disturbance. Inside the loop, the filter holds the control voltage still, which is exactly why the closed-loop test above says no. These numbers come from driving the block directly, with nothing holding it. So the supply rows say nothing about this PLL's supply rejection. They say something about the model.

9 Why a green testbench is worth believing

Generated verification has an obvious credibility problem. The same tool writes the model *and* writes the test. What stops it from writing a test its own model is bound to pass? Three rules.

The right answers come from SPICE, never from the model. Every expected value in every generated environment is a golden value: a SPICE measurement of your circuit. Check a model against the equations it implements and it passes however wrong it is. So spice2rnm never does that.

Every environment is proven able to fail. The tool breaks the generated model on purpose and confirms the test goes red. It also *measures* the smallest break the test still catches, rather than just claiming there is one. The example below is the duty-cycle corrector's published environment, the one judged against its specification's 1.0 pp requirement (pp means percentage points of duty cycle). The same testbench was run seven times, each time with the model broken in one known way:

Injected fault	DC section	duty section	worst duty error
none	46 pass	14 pass	0.43 pp
comparator offset, +5 mV	1 fails	pass	0.43 pp
comparator offset, +20 mV	1 fails	pass	0.73 pp
comparator offset, +50 mV	2 fail	pass	0.77 pp
dynamics slowed 2×	pass	pass	0.65 pp
dynamics slowed 10×	pass	pass	0.87 pp
dynamics slowed 50×	pass	9 fail	14.3 pp

The table says two things. First, the DC section is blind to *any* error in how fast the block responds. Slow the model down fiftyfold and all 46 of its DC checks still pass. Yet DC checks are all that many hand-written environments contain. The duty section exists to cover that blind spot, and it is the only check here that catches the slowed model.

Second, the table shows where the test stops catching things, and why. A tenfold slow-down shifts the output duty by 0.87 pp. That is under the 1 pp the specification allows, so the test passes. The tool's own default limit for this environment is 0.5 pp, which would have failed it.

An environment built from a specification is exactly as strict as the specification, and no stricter. That is on purpose. A requirement is what the circuit is held to, and an environment that quietly applied a tighter limit would report failures the design does not have. What the fault sweep adds is the number. You now know that this specification cannot catch a tenfold error in the block's speed, and you can decide whether that is the requirement you meant to write.

The test is checked in both directions. A test that fails everything is as useless as one that passes everything. So the same sweep also confirms that small, acceptable errors do *not* trip the environment. The unbroken model passes, and a 5 mV offset moves the duty cycle by nothing the duty section should object to.

The sweep is re-run against the shipped model whenever the environment changes, so the table always describes the environment you get. That is why this table differs from the earlier one, which was measured against the default limit. The claims stay measured results instead of drifting into folklore nobody rechecks.

10 Reading the written specification

A netlist says what a circuit *is*. It says nothing about what the circuit is *for*, what rate it has to run at, or how much error you are willing to accept. You can hand the tool those facts as a plain-language document: a datasheet excerpt, a requirements page. The tool reads out the facts that shape the verification:

From the specification	Used for
operating clock range	where the environment drives the block
accepted input duty range	the stimulus sweep
duty accuracy requirement	the pass/fail tolerance
block function	which environment is generated

Case study 2 is run this way. It reads `specs/dcc_spec.md` from the repository. Its duty-cycle environment drives 35–65% input duty at the clock rates the document names, and judges the result against the document's own 1.0 pp requirement. Result: 14 checks, 0 failed, worst 0.428 pp. PASS

The specification is cross-checked, never trusted. Every fact taken from it is compared with what the circuit was measured to do, and the measurement wins. When the two disagree, the tool reports the disagreement:

```
SPEC CONFLICT: the spec says this block operates up to 4e+08 Hz, but
the fitted model is only valid below 1.195e+08 Hz -- 3.3x lower.
```

That is a gap in coverage: the requirement asks for more than the measurements cover. It is caught when the model is generated, not months later at silicon correlation. Reviewing the model by hand cannot find it, because no single document holds both the requirement and the measurement.

11 Joining an existing model library

A team that already has real-number models also has house rules: which user-defined nettype carries analog values, which package declares that nettype, and what port discipline the models use. Generated files that import a nettype the tool made up do not join that library. They sit beside it, the two net schemes cannot be wired together, and the customer pays to bridge them.

Point `--style-from` at the library and the output follows its conventions. The environment's analog nets use the library's nettype. Its files import the library's package. The tool generates no types package of its own. The generated run script compiles the library's package file. And if the library's models use `wreal` ports, the generated model gets wreal ports too, without being asked.

One thing is deliberately not copied: the library's timescale. It is reported instead. Generated delays are computed femtosecond values, and timescale applies per compilation unit, so the two timescale directives work together unchanged.

Reading a house style takes judgement, so the tool reads it and then checks itself. A library may declare several nettypes when only one of them is the current standard, and often the only thing saying which is a comment. We measured this on a library that declares two, where the deprecated one happens to sort first alphabetically:

mechanical scan	picks the deprecated nettype, and warns
language-model reading	picks the current one, quoting the deprecation notice
verification	re-checks the choice against the named file

Here is why the language model said what it said, word for word from that run: *"line 15 marks `aaa_legacy_wire` deprecated ("kept only so ancient testbenches still compile. Do NOT use in new code"), ... both models declare all analog ports as `acme_ewire`, confirming this is the live convention."* None of that is taken on trust. The nettype, the package and the file it names are all checked against the file before anything uses them. If the file does not back the claim, the mechanical scan's answer stands and the run says that it fell back. And where the tool has a *measurement* of its own, here a count of which port discipline the library's models actually use, the count wins outright. Anything that disagrees with it is recorded, not adopted.

The result is an environment that runs on the library's own net type, and the generated files are published so you can look at them. The showcase carries the same filter twice. `lpf2/` was generated normally. `lpf2_house/` was generated against `house_lib/`, a small hand-written library standing in for a customer's own. Diff the two directories and the whole effect shows up:

	`lpf2/`	`lpf2_house/`
types package	generated	**not generated**
nets	`rc_lpf2_wire`	`ng_anet`
imports	the generated package	`ng_ams_pkg`
run script	compiles the generated package	compiles the house package
model ports	plain `real`	`ng_anet`, and the environment instantiates it
the model	byte-identical	
the scoreboard	byte-identical	

Following the house style changes the boundary of the model and nothing else. The published environment runs where it sits, against the library next to it: 49 DC and 15 AC checks, 0 failed, the same result as the default environment.

Look at what the environment instantiates in the second column. When the model ships inside a port wrapper, the generated testbench drives that wrapper, not the core inside it, and the wrapper's file joins the compile list. So the evidence covers what you actually ship, not just one piece of it. In separate measurements, a library whose nettype resolves multiple drivers by averaging instead of summing also conforms, and passes the same way. A library using `wreal` ports gets wreal ports on the model instead. Without `--style-from`, or pointed at a directory with nothing recognisable in it, nothing changes, and the run says so.

12 AI where it helps, physics where it decides

spice2rnm asks a language model four kinds of question: where one block ends and the next begins, what a block is for, what a written specification requires, and what conventions an existing model library follows (Section 11). Every one of those is a question about *intent*. Not one of them is a question about physics.

The rule is short. **Intent from the language model, physics from the measurement.** Every suggestion is checked against the measured circuit before anything uses it. If it does not match, it is thrown away, the run carries on with the measured data, and the run says that it did. With no language model reachable at all, every AI feature switches off cleanly and the pipeline runs on measurement alone.

Which of those two happened is recorded in the run's result, not just in its log. Each verdict records the metric it was reached with. So a reader coming back months later can tell a check that ran as asked from one where the AI feature bowed out and the tool fell back to measurement. Without that record, you would have to take it on trust that a passing result used the feature you asked for.

13 Scope and requirements

- **What goes in**: SPICE netlists, flat or with subcircuits. Optionally, a plain-language specification, and an existing RNM library whose conventions the generated models should copy.
- **What comes out**: SystemVerilog real-number models, with whichever port type you ask for. `wreal` by default, plain `real` for a flow that has no wreal support, or your own library's nettype. Also UVM-MS 1.0 environments, and equivalence reports with the waveforms kept.
- **What you need installed**: a SPICE simulator to do the measuring, and a digital simulator that can run real-number models (with UVM support, for the generated environments). The models and environments are standard SystemVerilog, tied to no vendor, so either simulator can be the one you already use.

- **Which blocks**: single-input single-output blocks, and blocks built out of those, including current-mode blocks that share a node. Stacking blocks into a hierarchy buys you structure and costs you a little accuracy. The equivalence report prints the stacked figure separately from the per-block figures, so you can see what the stacking cost.

- **Jitter: the transfer is checked, the oscillator's own noise is measured.** Both are measured. They are different jobs.

  Jitter transfer is what the model does with a disturbance you hand it. This one gets a pass/fail limit. The disturbance is injected into the transistors in SPICE, the very same disturbance is replayed into the model, and the period jitter of the two is compared. It is done on the control input and on the supply. Section 8.3 has the numbers: six points over a tenfold range, worst ratio 0.940 against a 0.25 tolerance. Nobody asked for this check. The pipeline ran it by itself, because the netlist declares noise on a net the ring oscillator is wired to. It is a real check with a real verdict, and it fails a model whose transfer is wrong.

  The oscillator's own noise is measured through its impulse sensitivity function (ISF), $\Gamma(\phi)$. That is a mouthful for a simple idea. Give the oscillator a tiny kick of charge at point ϕ of its cycle and see how far the kick shifts its phase for good. Kick it at one point and almost nothing happens. Kick it at another and the shift is large. $\Gamma(\phi)$ is that sensitivity, point by point around the cycle. The pipeline measures it without being asked: it injects a narrow charge pulse at each phase and measures the lasting shift.[2]

  On the reference ring, the rms value Γ_{rms} comes out at 4.04×10^{12} rad/C from the automated sweep, against 4.33×10^{12} measured by hand. The two agree to 7%. Γ changes sign across the cycle, as a ring oscillator's has to, and halves when the injected charge halves.

  Multiply Γ_{rms} by a device noise density and you get a jitter: 0.236 to 0.578 ps rms here, or -114.5 to -106.7 dBc/Hz at 1 MHz. It comes as a range because two of its inputs are themselves ranges: the transistor channel-noise factor, and the way the noise densities of the individual nodes add up. The model carries this jitter as a parameter, shipped set to zero. The figure is worked out from the measured ISF rather than compared against a SPICE transient, because ngspice does not simulate device noise in a transient run.

  Driven blocks jitter differently, and the models follow suit. An oscillator times each edge from its own previous edge. Its phase therefore wanders off on a random walk, and the timing error piles up. A buffer or a lag block times its output from an *input* event instead. Each trip through the block picks up its own independent random delay, and nothing piles up.[3] On that basis, the scheduled-lag models take a synchronous-jitter figure and nudge the input-to-output delay once per trip through the block. You supply that figure. It is not

[2] The method is Hajimiri, Limotyrakis and Lee, *Jitter and Phase Noise in Ring Oscillators*, IEEE JSSC 34(6), June 1999. Two details decide whether it works, and getting either wrong returns a plausible wrong answer rather than an error, so the tool enforces both. First, the pulse goes in differentially where the circuit is differential. Injected into one node of a differential ring it is mostly common mode, which the ring is built to reject, and the measurement then just reads its own noise floor however big the pulse. Second, linearity is checked at the phase where the ring is most sensitive, not at a convenient one. The same charge that moves the phase by 0.04 ps where the ring barely responds moves it by 52 ps at the peak.

[3] The distinction and both implementations are Kundert's, in *Modeling and Simulation of Jitter in Phase-Locked Loops*, AACD 1997. This model follows four of his points. Driven systems jitter without accumulating, while autonomous ones (free-running oscillators) have a phase difference that is "a random walk that is not bounded". The metric for both is $J = \sigma(T)$, the standard deviation of one period. Driven jitter is modelled by "randomly dithering the delay argument ... on every output transition". And with N updates per period, the per-update dither scales by $\sqrt{N}$, which is 1.414 for the two updates this model uses.

Each point was checked. Asked for 1% rms period jitter, the model produced 0.9998%. Its scaling constant is the $\sqrt{2}$ his listing specifies. Over 30,000 cycles, the variance of the accumulated deviation grew linearly with cycle count, at 1.004 ± 0.026 of the kJ^2 his random walk requires.

One setting is refused. At one sub-step per half cycle the scheduler cannot resolve a randomised threshold, so the model turns that setting down rather than report 39.6% jitter when 1% was asked for.

measured, and it cannot be: a SPICE transient of a netlist that declares no noise holds no delay variation to measure. The generated file says so. It also states the fixed extra latency the nudging costs, because the model's delay line can only look backwards in time, so an edge can be made early only relative to a delay that is already there.

Edge jitter is measured wherever the transistors in SPICE have clock edges. On the reference PLL's two clock-path buffers it is 3.98 ps and 3.60 ps. It is printed next to the skew, because jitter and skew go wrong in different ways. Edge jitter is reported for information, with no pass/fail limit, because there is no agreed threshold for it. Inventing one from a single design would be worse than just showing the number.

The system-level equivalence check also shows its jitter with no limit, and here the reason is the measurement rather than the model. The check's boundary log is sampled every 100 ps. The composed model's output crosses its threshold between samples, so a crossing can only be placed to the nearest sample. Calling that jitter would be measuring the sampling grid, not the circuit.

The generated UVM-MS environment has a monitor without that problem. It reacts to each change of the signal instead of sampling on a clock. It is compiled with a `1fs` timescale while the rest of the environment stays at `1ps`, which costs nothing elsewhere because SystemVerilog sets timescale per module. Run that environment on the composed PLL and it reports 12.70 ps rms period jitter over 1151 periods at the boundary. Over 116,012 boundary crossings, none of its system checks failed and one was inconclusive. The total number of checks is not quoted because it varies: one check exists only when the transistors settled measurably, which under a noisy supply they may or may not do.

That 12.70 ps is for information only too, for a specific reason. A PLL suppresses its own oscillator's jitter inside the loop bandwidth. A figure measured at the PLL boundary is therefore not the oscillator's own jitter, and nobody should expect the two to agree. The 11.80 ps that this netlist's declared supply noise produces at `ro_vco` is a different measurement of a different quantity. Both figures above come from the same run, which earlier versions of this paper could not say. The gap between them is 7.6% here, computed from the two figures above rather than carried from an earlier run. Each is one run's draw from a random spread – the oscillator's own figure carries ±5% over the two hundred cycles it was measured on – and neither is evidence about the other. Both are reproducible rather than quoted: the oscillator's is `vco_results.ro_vco.intrinsic_jitter.rms_s` in the published `result.json`, and the boundary figure is the last line of the environment's own saved output in `showcase/pll_cosim/uvm_ms_runs/cosim.log`.

This validation goes further than published practice. The gap named above, no intrinsic device noise in the model, is one published practice shares. Analog Devices' production RNM suite for a GHz transceiver validates its PLL on lock time and frequency error, the same quantities checked in Section 8. It says that "model validation remains an area for improvement in the real number modeling flow", and lists frequency-domain performance as future work.[4]

A What the routes measure

A route is the method the tool picks for measuring a block. The case studies name each block's route and say why that block needs it. This appendix says what the routes actually measure.

[4] J. Dudek, J. Nekl and K. O'Donoghue (Analog Devices), *Real Number Modeling of RF Circuits*, DVCon proceedings, `https://dvcon-proceedings.org/wp-content/uploads/real-number-modeling-of-rf-circuits.pdf`. The paper notes that because RNMs exist to enable system-level testing, "the requirements for model validation are greatly relaxed in this environment".

It uses the reference PLL's own numbers, not a made-up example. You do not need it to read the results. It is here if you want to check them.

A.1 Why an oscillator is compared as a rate

Most blocks are checked by laying two output waveforms on top of each other and subtracting. That is the first row of the table in Section 8. An oscillator, the fourth row, cannot be checked that way at all, and the reason is the whole case for giving it a route of its own.

Picture a perfect model, running at exactly the frequency the transistors run at. The two start in step. Nothing holds them there. Let their frequencies differ by one part in 10^5 and inside a millisecond they have slid a full cycle apart. From then on the model sits at one supply rail whenever the circuit sits at the other. Subtract the waveforms and you get full-swing error everywhere. The perfect model scores as completely wrong.

This is not a thought experiment. A milder version of it appears earlier in this paper. The two clock buffers in case study 4 report a worst-instant error of 99.9% of full swing, because an edge lands slightly late. In a buffer that bad instant lasts a moment per cycle. In an oscillator it would be the permanent state of affairs.

So the oscillator route subtracts nothing. It compares rates instead. Set the control voltage. Run the transistors in SPICE and count cycles. Run the model at the same control voltage and count its cycles. Compare the two frequencies:

set `cont` to	1.9000 V
SPICE: count cycles, divide	f_{spice}
model: same control, count	f_{model}
score	$\lvert f_{\text{model}} - f_{\text{spice}} \rvert / f_{\text{spice}} = 0.024\%$
pass/fail limit	2%

Phase is ignored on purpose. The model never claims to reproduce it: what it puts out is an ideal square wave. The real ring's finite edge rates were already accounted for when its frequency was measured by FFT.

Every verdict records which row of this table it came from. So months later you can still tell which number decided the result, and which numbers were only noted beside it.

A.2 The tuning curve

For an oscillator the tuning curve is not a description of the model. It *is* the model. The route measures a table with one row per control voltage. Each row is its own transient run of the transistors in SPICE, with the control voltage pinned and the resulting oscillation timed:

`cont` (V)	frequency (MHz)
0.495	804.47
⋮	⋮
1.750	438.73
1.850	399.86
⋮	⋮
2.805	16.74

The model looks its frequency up in this table and interpolates between rows. That is why its checks come in three kinds:

- **At the table's own 15 rows.** The model is exact here by construction. The agreement (0.001%) only shows that the table was copied across correctly.

- **At 5 control voltages *held back* from the table.** Here the model has to interpolate, and it is compared against fresh SPICE runs. This number, 0.024%, is the only one of the three that says anything about accuracy.
- **At 9 voltages far outside where the closed loop goes.** Out there the error reaches 4.987%. It is reported for information, with no pass/fail limit, because the loop never visits.

Section 6.4 reads the same table backwards. Ask it for 400 MHz and it answers 1.8496 V. The assembled loop is then required to settle at that control voltage.

A.3 Control states

A control state is one combination of the digital inputs driving a block. The charge pump has two of them, `pdn` and `pupb`, and they are driven *independently*. So it has four states rather than one continuous knob. Each state gets its own measured table of current against output voltage, $I(V)$: 12 points from 0 to 3.3 V, with 11 more held back for checking.

state	(pdn,pupb)	i at 1.80 V	at 3.3 V	what it does
00	$(0,0)$	+7.239 mA	≈ 0	sources: charges the filter
01	$(0,1)$	−1.9 nA	−4.0 nA	high-Z: off
10	$(1,0)$	+2.798 mA	−5.078 mA	both on, fighting
11	$(1,1)$	−4.441 mA	−5.078 mA	sinks: discharges it

Why one table per state, not one fitted curve. The gates are digital. Each one is either on or off, so there is no continuous knob to fit a curve against. What there is instead is a small, fixed set of modes. Inside a mode, the current depends only on the output voltage, because how much current you can push into a node depends on the voltage already sitting there. That is what $I(V)$ means here, and why there are four tables.

It is also why the default route cannot model this pump. The pipeline works that out from the shape of the circuit, not from the block's name. An inverter ties its two gates together, so one input drives both and there is a single thing to fit against. In the pump the two gates are separate signals. That makes states 01 and 10 reachable, and an inverter has neither. State 01 is the idle one, drawing nanoamps. State 10 has both transistors conducting at once.

These four rows do work in two other places in this paper. First, the loop harness of Section 6.4 uses them to work out each gate's active level. It reads them at 1.849 V, where the loop settles, rather than at a table row. With `pupb` idle, `pdn` at 1 reaches state 11 and sinks 4.47 mA, so `pdn` is active high. With `pdn` idle, `pupb` at 0 reaches 00 and sources 7.103 mA, so `pupb` is active low. Second, the equivalence verdict names the state its worst error happened in: 0.5487% against a 2% limit, at state 00. An average over the four modes would hide which mode is the weakest.

A.4 The state space

A filter can be written down exactly, so its route needs to measure nothing at all. The standard way to write a linear dynamic system is a state space: a few variables whose *rates of change* depend on where they are now and on the input, $\dot{x} = Ax + Bu$, $y = Mx + Nu$. In an RC network the state variables are simply the capacitor voltages. Here is the netlist:

```
.subckt lpfilt dra vout
R1 dra  int1 300        C1 c1   0  10n
R2 int1 c1   200        C2 int1 0   5n
R3 int1 vout 50         C3 vout 0  0.5n
.ends
```

Three capacitors, so three states: the voltages on `c1`, `int1` and `vout`. The input is the current the charge pump pushes into `dra`. The output is `vout`. When this paper calls the route "exact and needs no fitting", it means every coefficient in the generated model is worked out from numbers written on those six lines:

generated coefficient	**is**	**which is**
500,000	$1/(R_2C_1) = 1/(200 \times 10\,\mathrm{n})$	the `c1` pole
40,000,000	$1/(R_3C_3) = 1/(50 \times 0.5\,\mathrm{n})$	the `vout` pole
1,000,000	$1/(R_2C_2) = 1/(200 \times 5\,\mathrm{n})$	`c1` into `int1`
4,000,000	$1/(R_3C_2) = 1/(50 \times 5\,\mathrm{n})$	`vout` into `int1`
200,000,000	$1/C_2 = 1/5\,\mathrm{n}$	the injected current

Not one of these came from a simulation. Compare the inverter in the same netlist: there the tool ran SPICE 33 times, fitted a curve, and reports an rms fit error. Here there is nothing to fit, so there is nothing to get wrong. Kirchhoff's laws and the netlist settle the answer between them. The comparison can still catch a code-generation bug, meaning a generated file that disagrees with the matrices it was built from. That is what its 0.019% measures.

Two details of the generated file come out of the same reasoning. First, `dra` is *not* a state, because it has no capacitance. Its voltage is set at every instant by the resistors and the injected current, so it is solved algebraically at each time step, together with the pump that drives it. Second, the filter has no DC path to ground. A steady injected current therefore ramps the output up forever and never settles. That is exactly what a charge pump driving a capacitive filter does, and it is why the filter is modelled as a state space rather than as a DC gain.

Veylon Systems

© 2026 Veylon Systems. spice2rnm is a Veylon Systems product. Every figure in this document was produced by the shipped tool from its bundled demonstration circuits, most recently on 2026-09-22. The demonstration suite at `https://github.com/vrajeshprakhya/spice2rnm-demos` regenerates them in one command.